Advanced Computer-Aided VLSI System Design

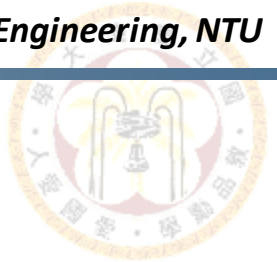
Chapter 6. Design for Testability

Lecturer: Can-Lin Wu

Graduate Institute of Electronics Engineering, National Taiwan University



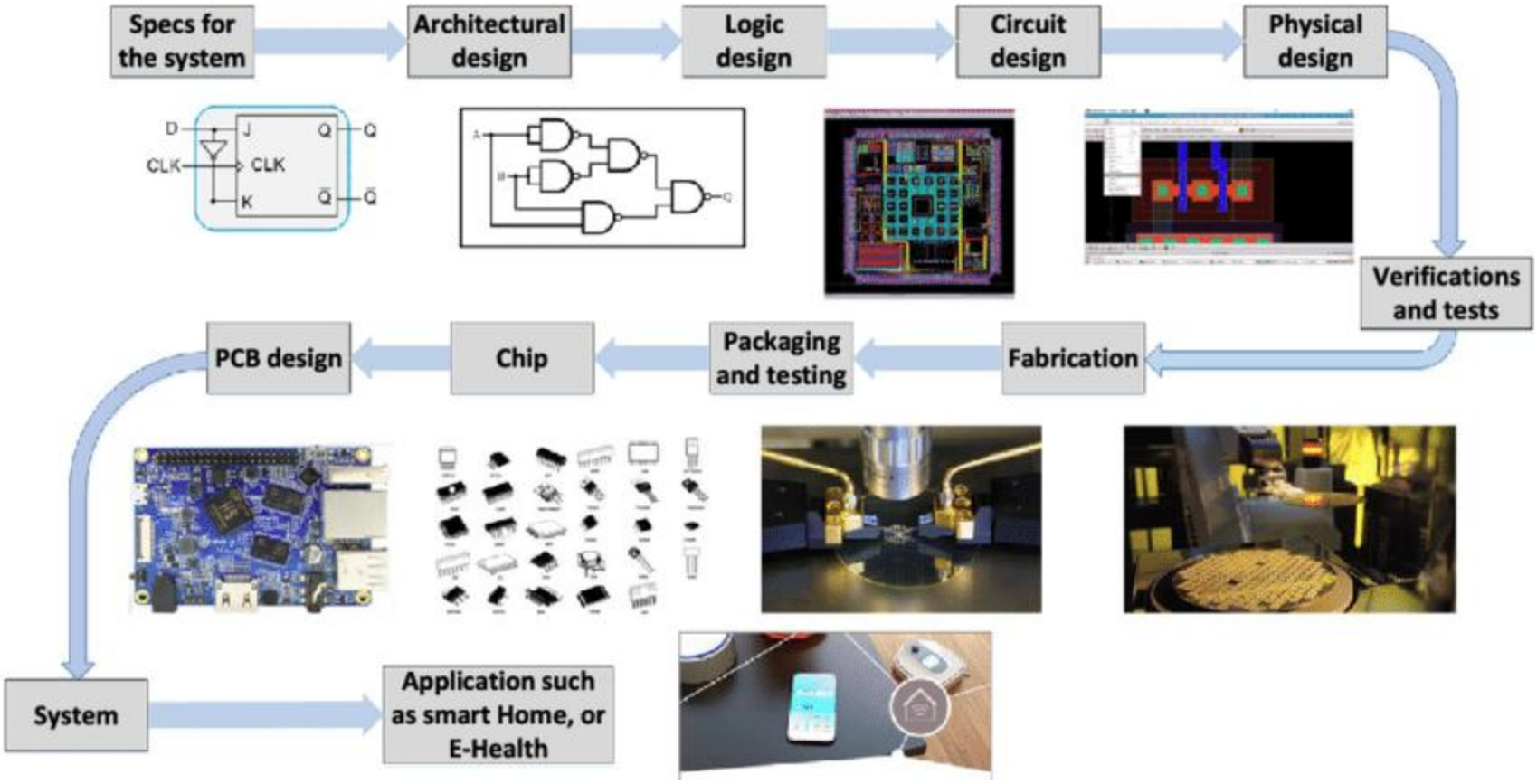
NTU GIEE



Outline

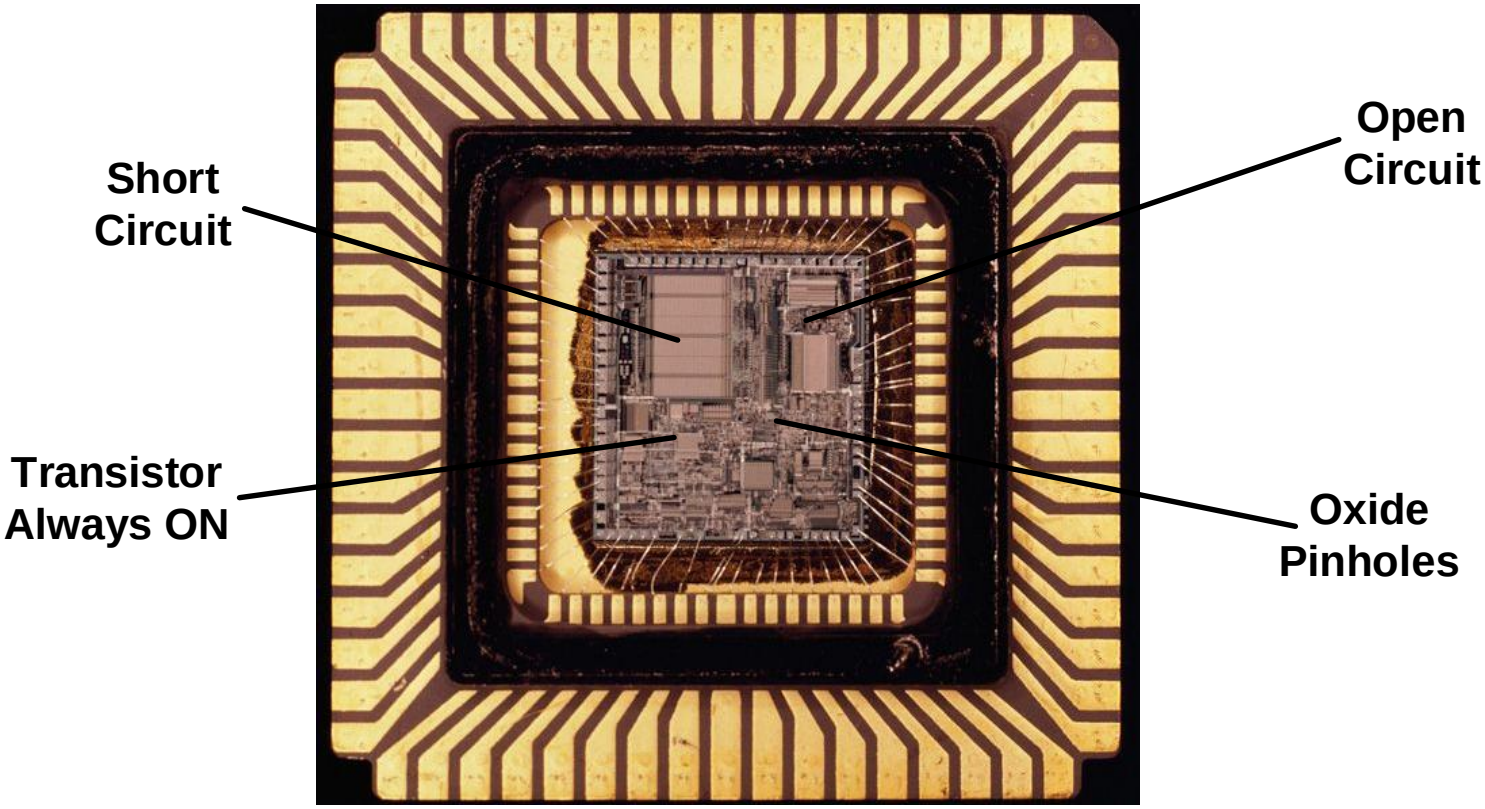
- Introduction
- Fault Modeling
- Design for Testability
- Test Generation
- Built-In Self-Test

Chip Design and Manufacturing Flow



Physical Defect

- A on-chip flaw introduced during fabrication or packaging of an individual ASIC that makes the device malfunction
- Testing is needed to ensure ASIC's correctness and reliability

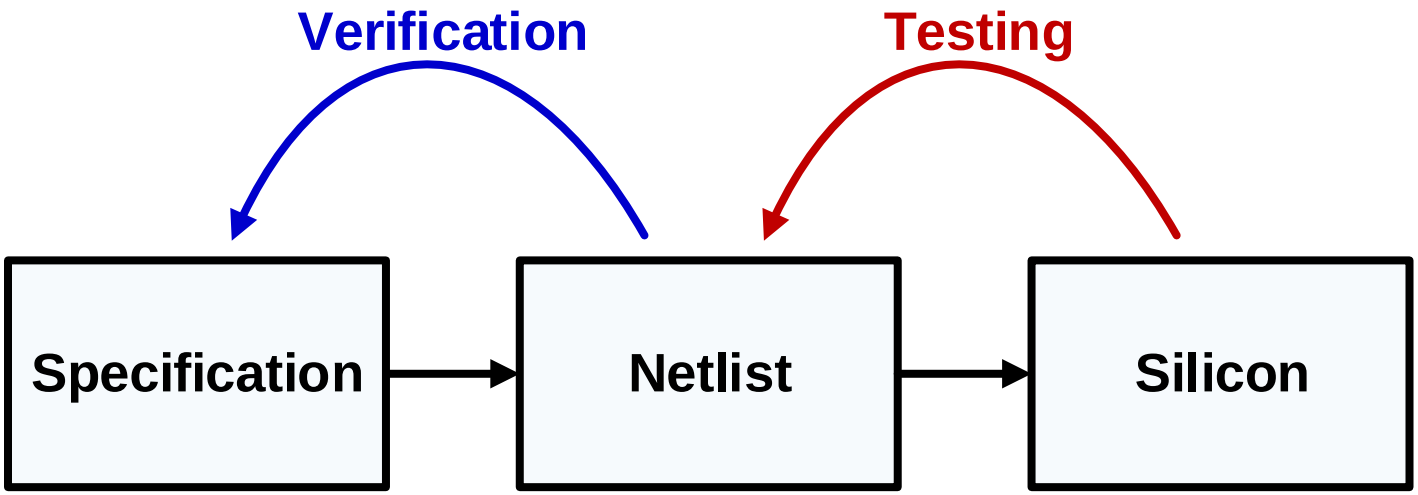


What is Testing

- Testing is a process of determining whether a device is function correctly or not
- Type of testing: on wafer test, characterization test, production test, burn-in test, diagnostic test…
- Test items: function test, structural test, parametric test, at-speed test, leakage test, UV test, ESD test…
  - Function test: verify functionality
  - Structural test: verify manufacturability
  - Parametric test: verify AC and DC parameters
  - At-speed test: verify performance
  - Leakage test: defects may cause high leakage current

Different between Verification and Testing

- VLSI verification: focus on confirming that a design behaves correctly according to its specifications
- VLSI testing: ensure that manufactured integrated circuits meet quality standards and operate correctly under different operating conditions

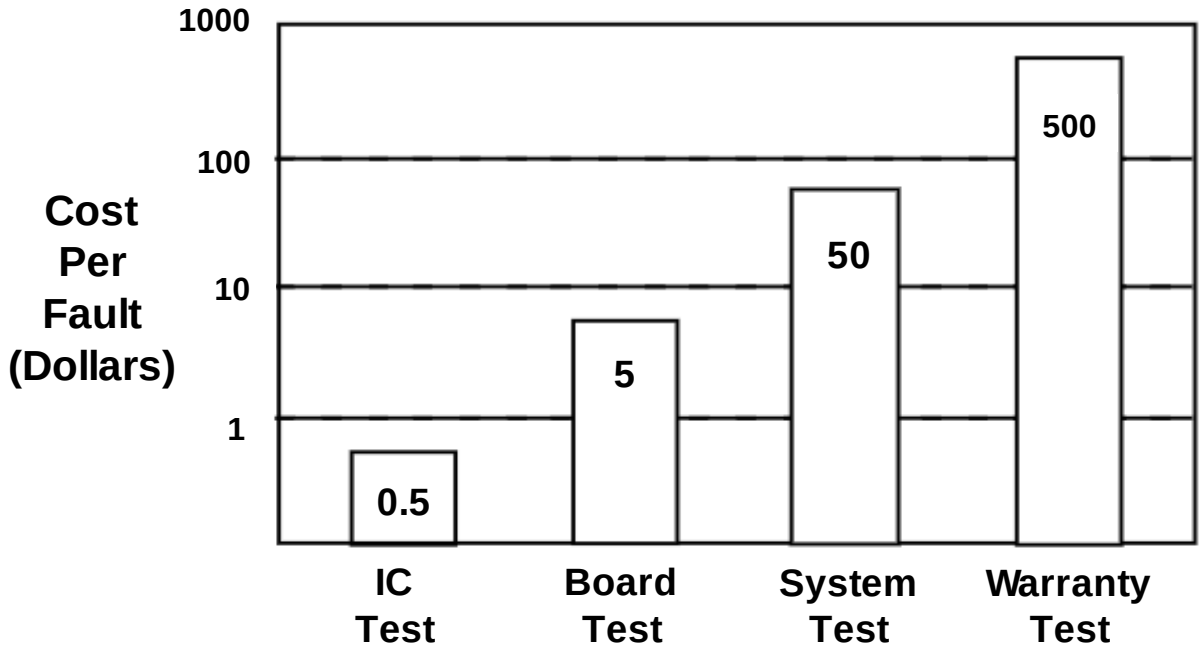


Purpose of Testing

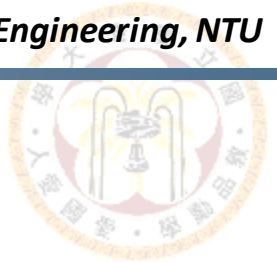
- Ensure that ICs are reliable, functional, and meet quality standards
  - **Quality assurance**: ensure defect-free chips reach the users
  - **Cost reduction**: detecting and addressing faults early reduces the cost of rework and scrap
  - **Reliability**: enhance system reliability and longevity
  - **Time-to-market**: streamlining the testing process helps in meeting market deadlines without compromising quality

Purpose of Testing

- Cost of repair
  - Goes up by an order of magnitude each step away from the fab. line



B. Davis, *The Economics of Automatic Testing*, McGRAW-HILL, 1982.



Yield and Fault Coverage

- Yield (Y) is the ratio of the number of good dies per wafer to the number of dies per wafer
- Y = (# of good dies) / (# of all dies)
- Fault: A representation of a defect at the abstracted function level
- Fault coverage (FC) is the measure of the ability of a set of tests to detect a given class of faults that may occur on the device under test (DUT)
- FC = (# of detected faults) / (# of possible faults)

Defect Level

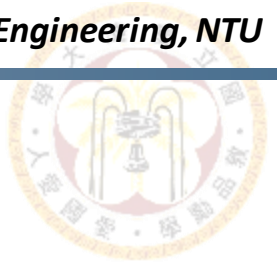
- Defect level (DL) is the fraction of devices that pass all the tests but still contain some faults
- $DL = 1 - Y^{(1 - FC)}$
- Defect level is the measured in terms of defective parts per million (dppm, ppm)

Yield	Fault Coverage	DPPM
90%	90%	10000
95%	90%	5000
99%	90%	1000
90%	90%	10000
90%	95%	5000
90%	99%	1000
90%	99.9%	100

Testing Problem

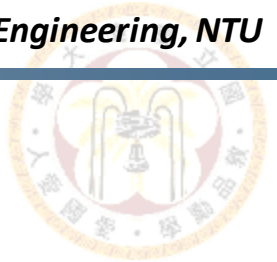
- Testing problem: given a set of faults in the device under test, how do we obtain a small number of test patterns which guarantees a high fault coverage?
- Test process
  - What faults to test? -> fault modeling
  - How are test pattern obtained? -> test pattern generation
  - How is test quality (fault coverage) measured? -> fault simulation
  - How are test vectors applied and results evaluated? -> Automatic Test Equipment (ATE)/ Built-In Self-Test (BIST)





Outline

- Introduction
- **Fault Modeling**
- Design for Testability
- Test Generation
- Built-In Self-Test

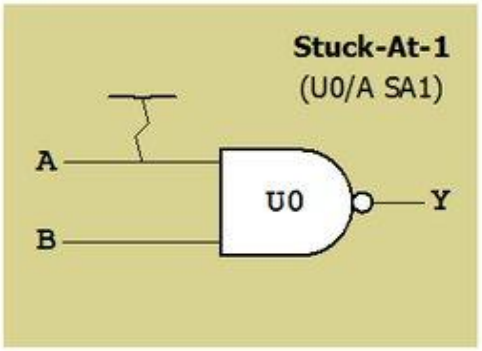


Fault Modeling

- Fault model: logical model representing the effects of a physical defect
- Common fault models
  - Single stuck-at fault model
  - Multiple stuck-at-faults model
  - Bridging faults model
  - Transistor stuck-short/open faults model
  - Other fault models

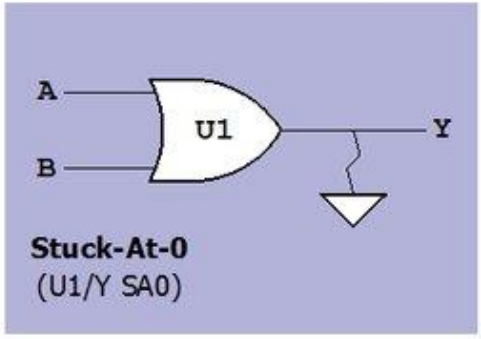
Single Stuck-At Fault Model

- A signal (gate input or output) in the circuit is fixed at logic 0 or logic 1 and independent of other signal values
- The most popular fault model
  - Reduce the complexity of testing



Input stuck at 1
independent of
value applied at A.

Output stuck at 0
independent of
values at A and B.

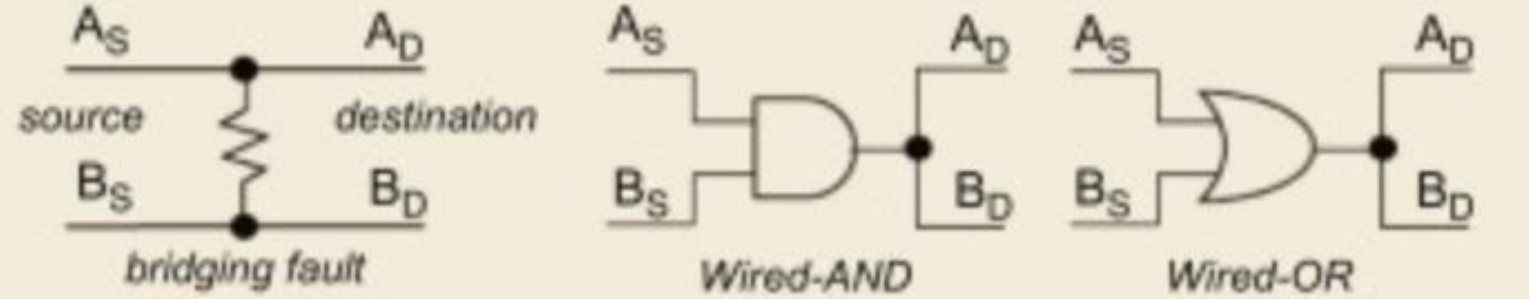


Multiple Stuck-At Faults Model

- More than one stuck-at fault in a circuit
- For a circuit with k signals
  - 2k single stuck-at faults
    - 2 (Stuck-at-1, stuck-at-0) * k
  - 3^k-1 multiple stuck-at faults
    - A signal can be stuck-at-1, stuck-at-0, or fault-free (3^k situations)
    - One situation is that all signal are fault-free (exclude this situation)
- Not scalable for large circuits
  - Single stuck-at fault model covers a large percentage of multiple stuck-at faults

Bridging Faults Model

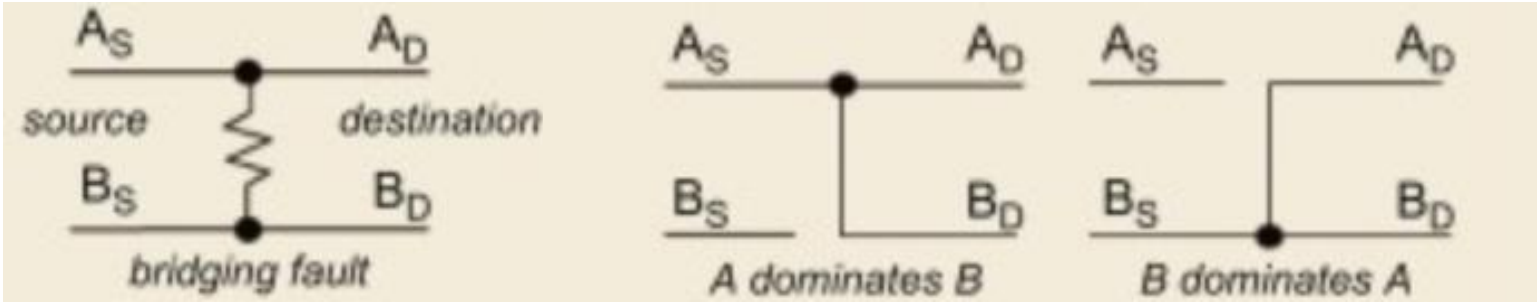
- Two or more distinct logic signals unintended shorted together and create wired logic
  - Wired-AND: the faulty value of A_D and B_D is taken to be the logical AND of A_S and B_S in the original fault-free circuit
  - Wired-OR: the faulty value of A_D and B_D is taken to be the logical OR of A_S and B_S in the original fault-free circuit



Source: https://www.sciencedirect.com

Bridging Faults Model

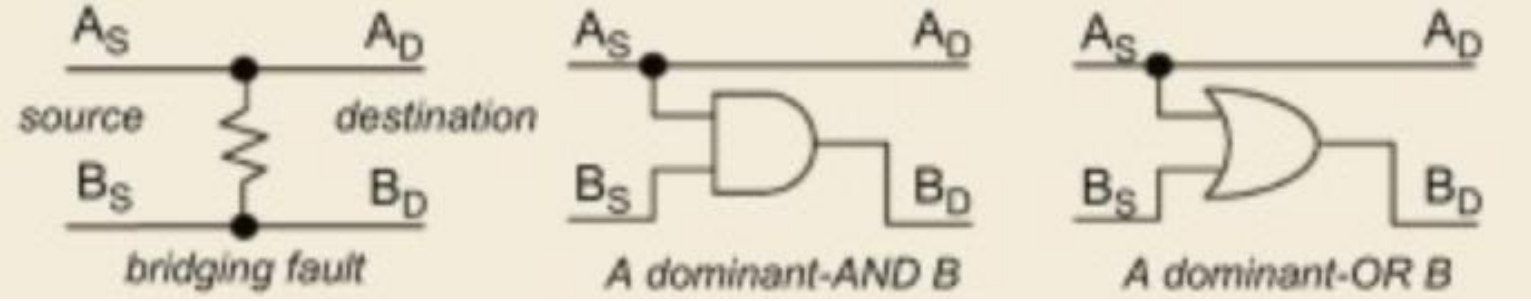
- Two or more distinct logic signals unintended shorted together and create wired logic
  - A dominates B: the faulty value for both A_D and B_D is taken to be the logic value of A in the fault-free circuit
  - B dominates A: the faulty value for both A_D and B_D is taken to be the logic value of B in the fault-free circuit



Source: https://www.sciencedirect.com

Bridging Faults Model

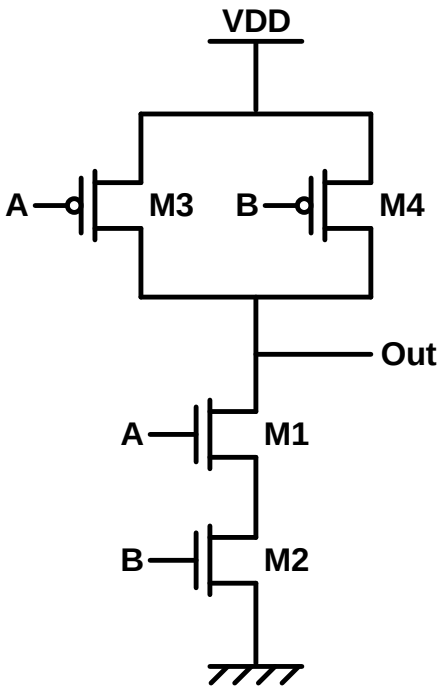
- Two or more distinct logic signals unintended shorted together and create wired logic
  - A dominant-AND B: the faulty value for B_D is taken to be the logical AND of A and B in the fault-free circuit
  - A dominant-OR B : the faulty value for B_D is taken to be the logical OR of A and B in the fault-free circuit



Source: https://www.sciencedirect.com

Transistor Stuck-Short/Open Faults Model

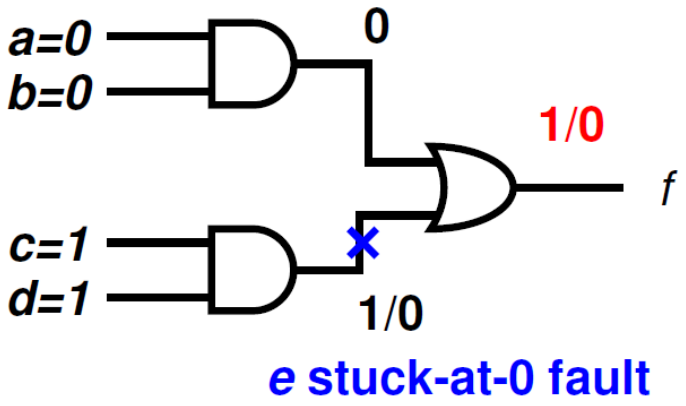
- Stuck-short (stuck-on): a transistor always conducts current
- Stuck-open (stuck-off) : a transistor never conducts current



AB	Out	M1 sO	M1 sS	M2 sO	M2 sS	M3 sO	M3 sS	M4 sO	M4 sS
00	VDD	VDD	VDD	VDD	VDD	VDD	VDD	VDD	VDD
01	VDD	VDD	Short Current	VDD	VDD	Z	VDD	VDD	VDD
10	VDD	VDD	VDD	VDD	Short Current	VDD	VDD	Z	VDD
11	0	Z	0	Z	0	0	Short Current	0	Short Current

Fault Detection

- A test pattern detects a fault if output of faulty circuit ≠ output of good circuit



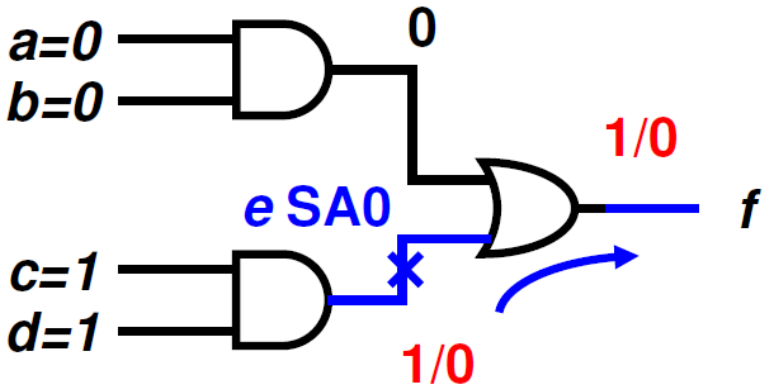
f = ab + cd

test pattern (a,b,c,d) = (0,0,1,1)
detects *e* stuck-at-0 fault

notation 1/0:
good value=1; faulty value =0

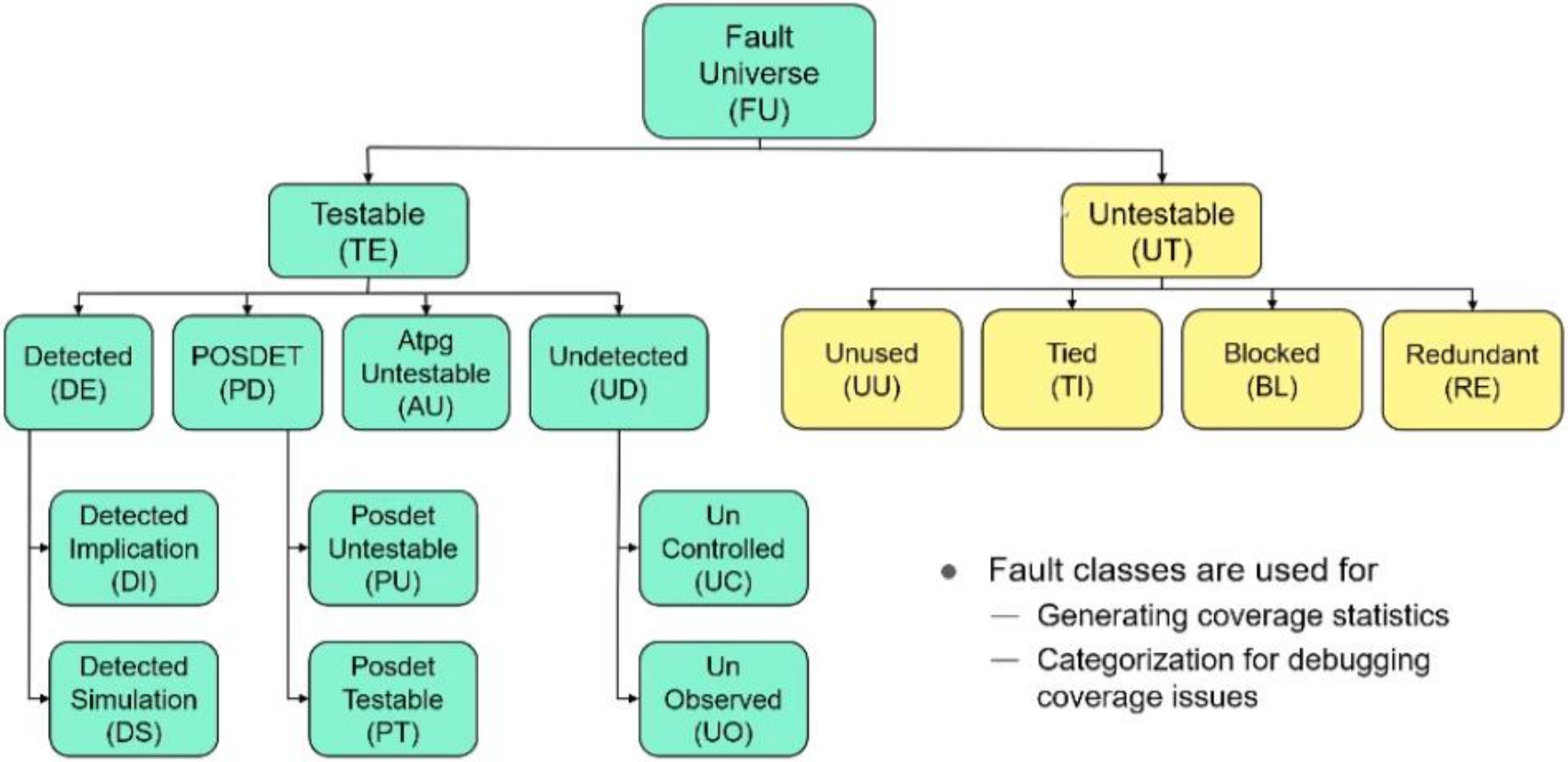
Fault Detection Requirement

- A fault is detected if two conditions are satisfied
  - Fault activation
    - Generate a fault effect at the site of the fault
  - Fault effect propagation (error propagation)
    - Propagate fault effect to any primary output



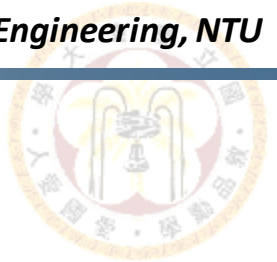
Fault activation: *c=d=1*
Fault Effect Propagation: *a=b=0*
e is sensitized to output
e-f is sensitized path

Fault Classes



- Fault classes are used for
  - Generating coverage statistics
  - Categorization for debugging coverage issues

Source: https://blog.csdn.net



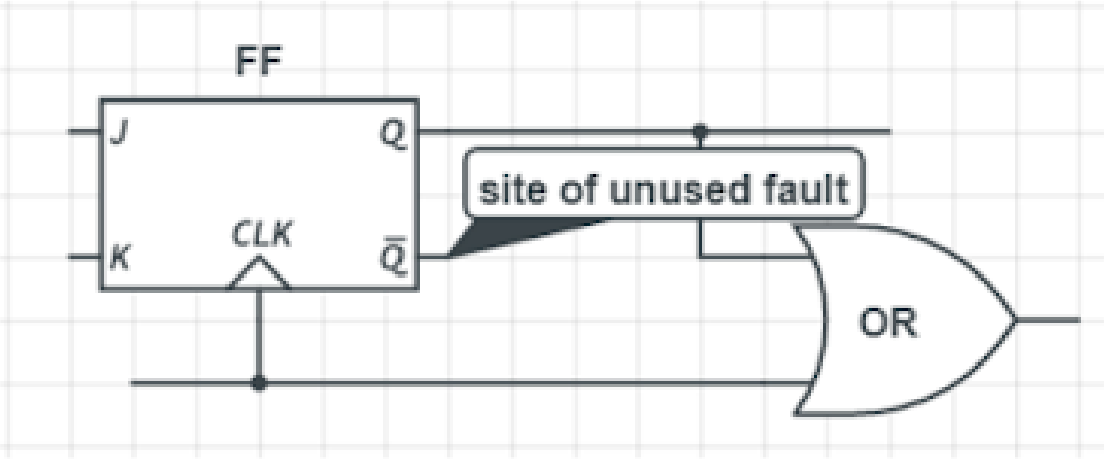
Fault Classes

- Untestable faults (undetectable faults): faults that cannot be detected by any test pattern (proven)
- Testable faults: faults that cannot be proven untestable
- Undetected faults: faults that are not detected by the given test set
  - Due to ATPG runtime limitation
    - Cannot prove it untestable
    - Cannot find test pattern either
  - No test pattern so far (may exist but not sure)

Untestable Faults ≠ Undetected Faults

Undetectable Faults: Unused (UU)

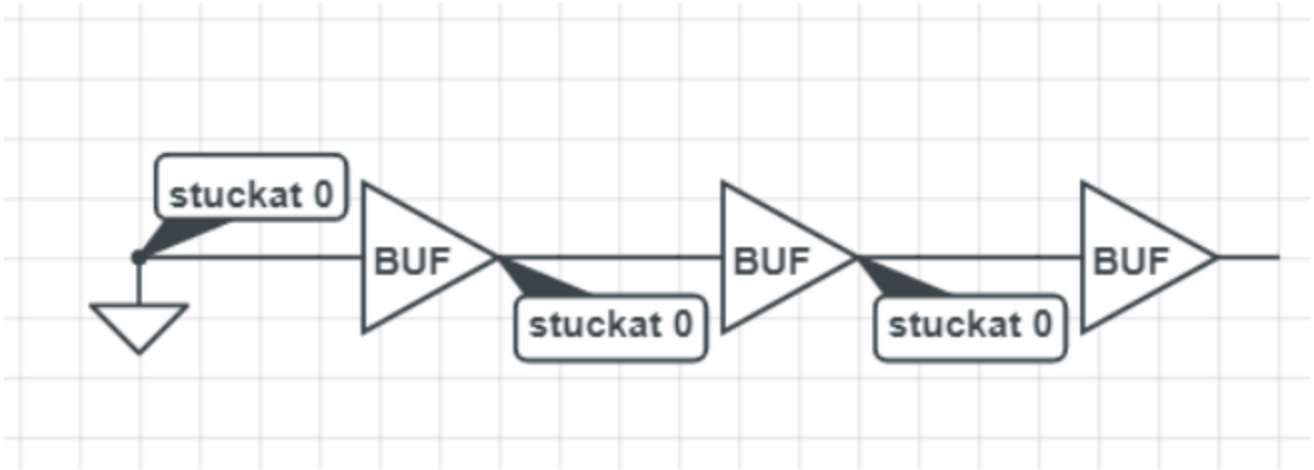
- Any floating pins not used in the design come under UU faults
- The unused faults class includes all the faults on circuit unconnected to any observation point



Source: https://www.vlsispace.com

Undetectable Faults: Tied (UT)

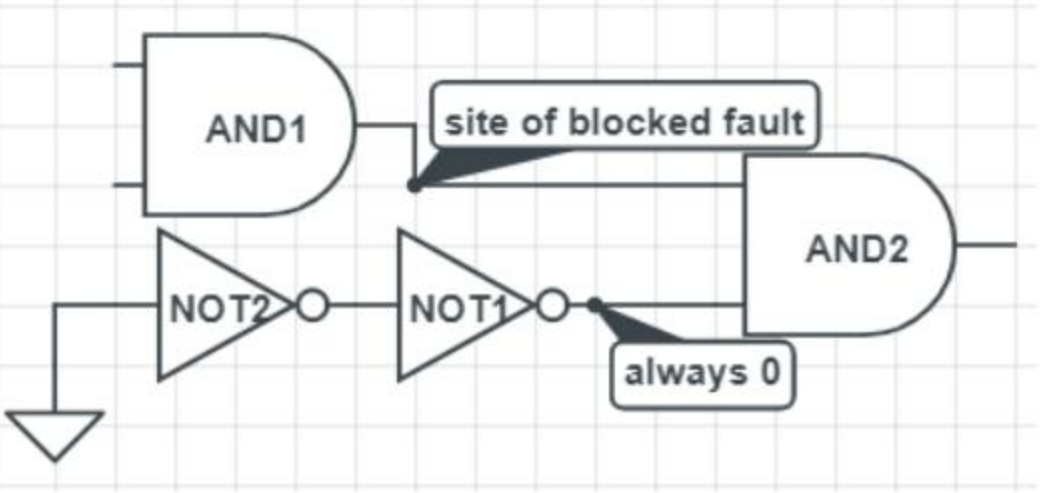
- This faults includes faults on gates where the point of the faults is tied to a value identical to the fault stuck value



Source: https://www.vlsispace.com

Undetectable Faults: Blocked (UB)

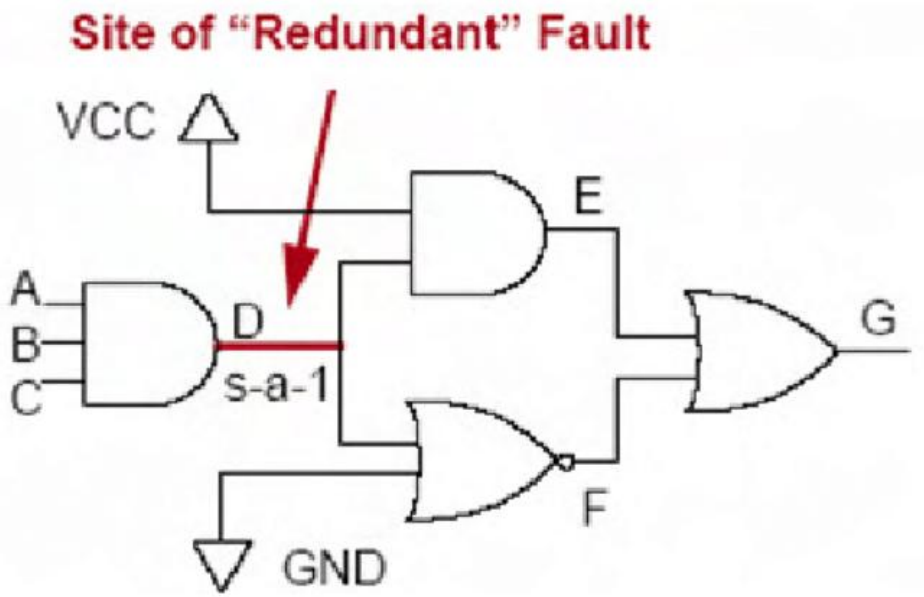
- Due to tied logic in the design, few faults are blocked and these are categories into blocked faults
- By adding the observable test point we can increase the coverage report



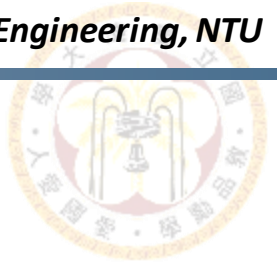
Source: https://www.vlsispace.com

Undetectable Faults: Redundant (UR)

- A fault that cannot be detected by any test pattern in the ATPG process because it doesn't affect the circuit's normal function



Source: https://www.researchgate.net



Evaluation Metrics

- Fault coverage (FC): measure quality of test set

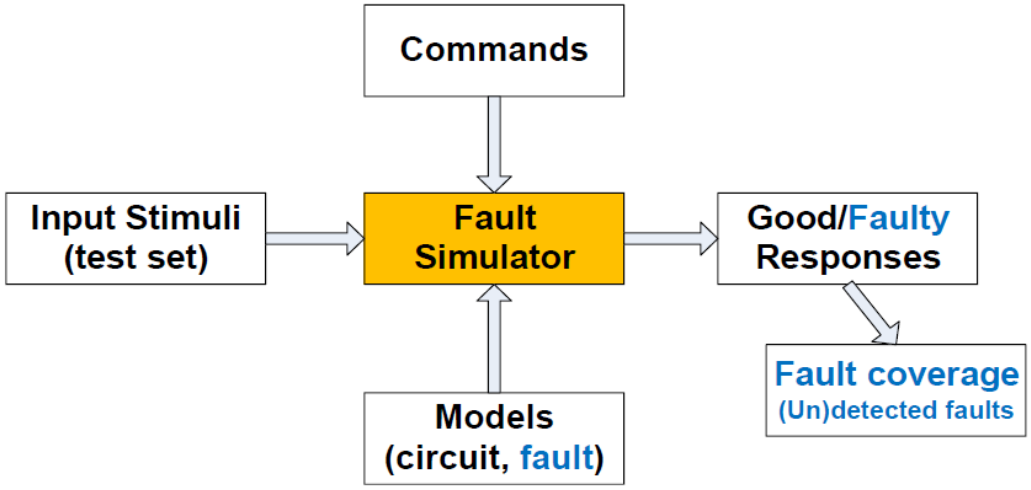
$$\text{Fault Coverage} = \frac{\text{\# of detected faults}}{\text{\# of total faults}}$$

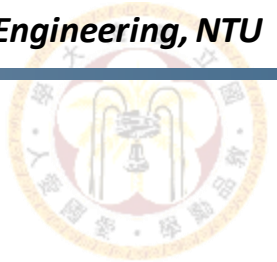
- Test coverage: measure quality of test set (testable faults only)

$$\text{Test Coverage} = \frac{\text{\# of detected faults}}{\text{\# of testable faults}}$$

Fault Simulation

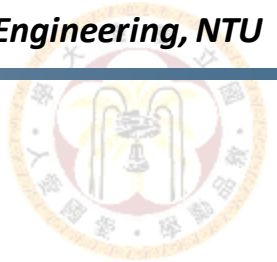
- To evaluate the quality of a test set (compute its fault coverage)
- Fault simulation techniques
  - Serial fault simulation
  - Parallel fault simulation
  - Deductive fault simulation
  - Concurrent fault simulation





Outline

- Introduction
- Fault Modeling
- **Design for Testability**
- Test Generation
- Built-In Self-Test



Design for Testability (DFT)

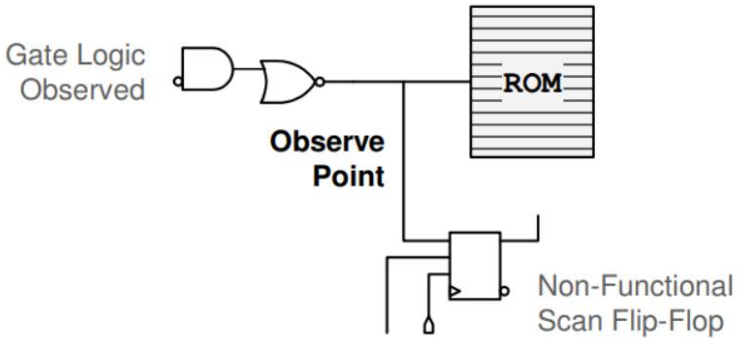
- Identify and resolve defects early in the development process
- DFT is a design technique to add circuitry to the chip, which improves the observability and controllability of internal nodes and increases the testability of all logic in the chip
- Importance of the DFT
  - Increased chip density: the number of transistors in a chip has grown significantly, which has made it more difficult to test all the components of a chip. DFT helps to address this problem by improving the testability of the chip
  - Manufacturing variability: DFT helps to detect and isolate defects that may result from manufacturing variability
  - Cost of testing: DFT helps to reduce the cost of testing by making the chip more testable

Design for Testability (DFT)

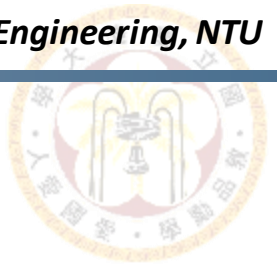
- Testability measure: metric to measure degree of difficulty to test a circuit
  - Controllability: degree of difficulty to control a logic signal to 0 or 1
  - Observability: degree of difficulty to observe the logic value of a signal
- DFT methods
  - Ad-hoc methods
  - Structured
    - Internal scan: full scan and partial scan
    - Boundary scan
    - Built-in self-test (BIST)
- Cost of DFT: pin count, area, performance, manpower, tool cost

DFT Methods

- Ad-Hoc Methods
  - Adding testability logic at designer's discretion
  - Example: create an observe point of ROM
  - Disadvantages
    - Cannot guarantee high fault coverage
    - Manual test generation
    - Design iterations required



- Structured DFT
  - Provide a more systematic and automatic approach to enhance design testability
  - The most common is the scan design technique, which modifies the internal sequential design into a scan design

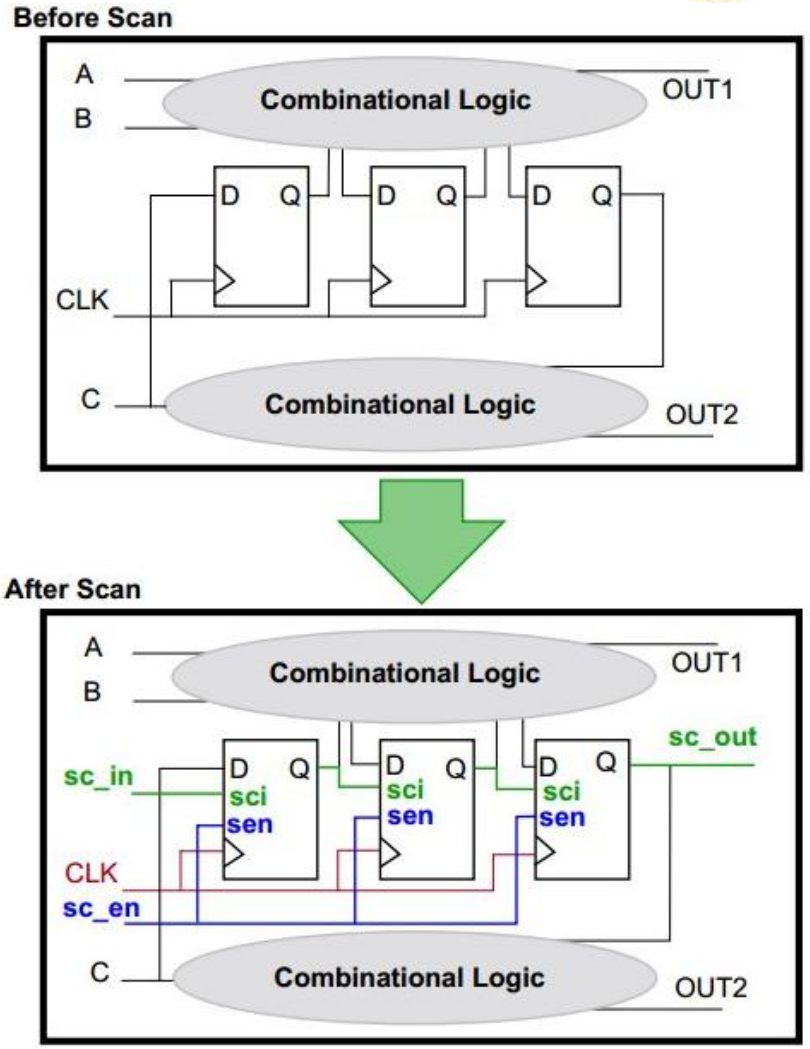


Scan Design

- Provide controllability and observability at internal flip-flops for testing
- Method
  - Add scan enable control signals to circuit
  - Connect flip-flops to form shift registers in test mode
  - Make inputs/outputs of the flip-flops in the shift register controllable and observable
- Type
  - Internal scan
    - Full scan: all flip-flops in the circuit are connected in a scan chain
    - Partial scan: only a subset of the flip-flops in the circuit are connected in a scan chain
  - Boundary scan

Scan Design

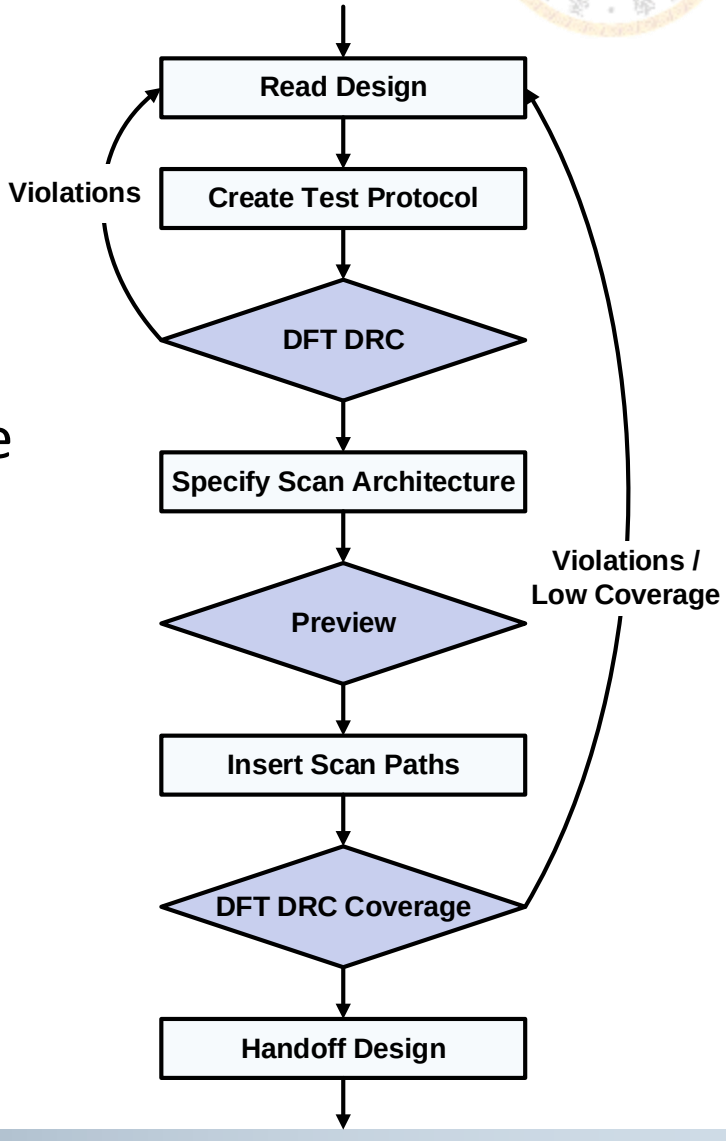
- Connect the flip-flops to form a long serial shift register
  - The internal state of the circuit can be observed and controlled by shifting (scanning) out the contents of the flip-flops
  - The shift register is called a scan path
- Two modes of operation
  - Normal mode
  - Test mode



[Source]: https://www.quora.com/What-is-a-scan-insertion-in-DFT

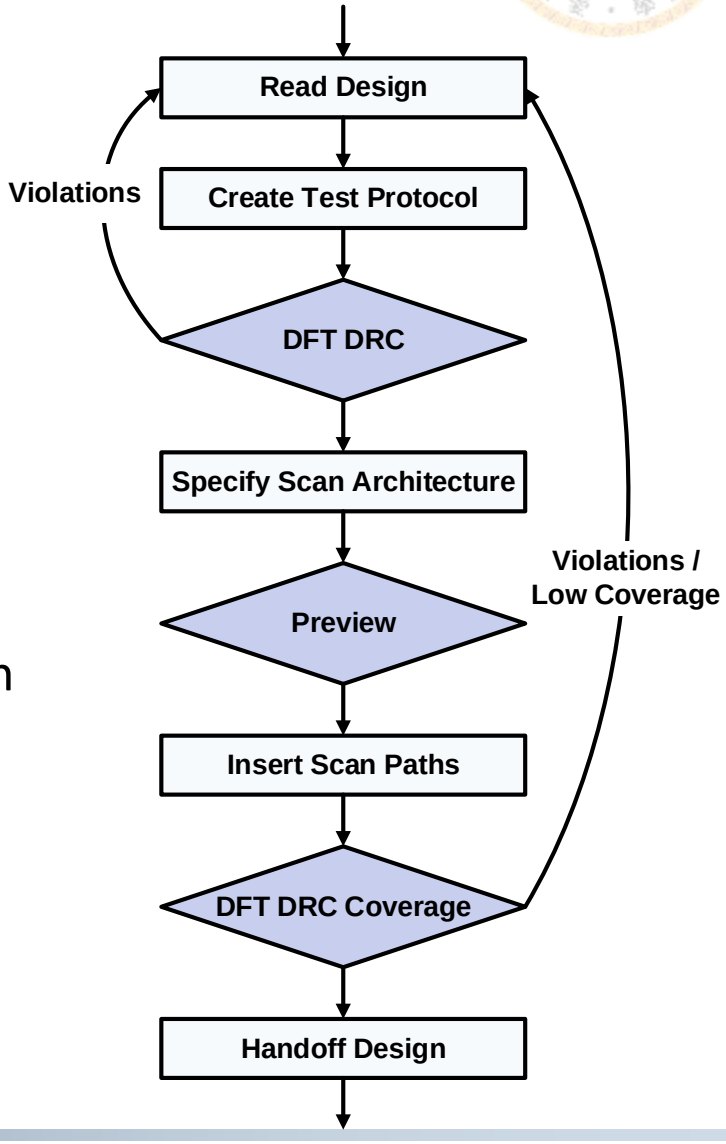
Scan Insertion Flow

- Read Design
  - Read a mapped design
    - read_ddc, read_verilog
- Create Test Protocol
  - The test protocol describes how the design operates in scan mode
    - set_dft_signal, create_test_protocal



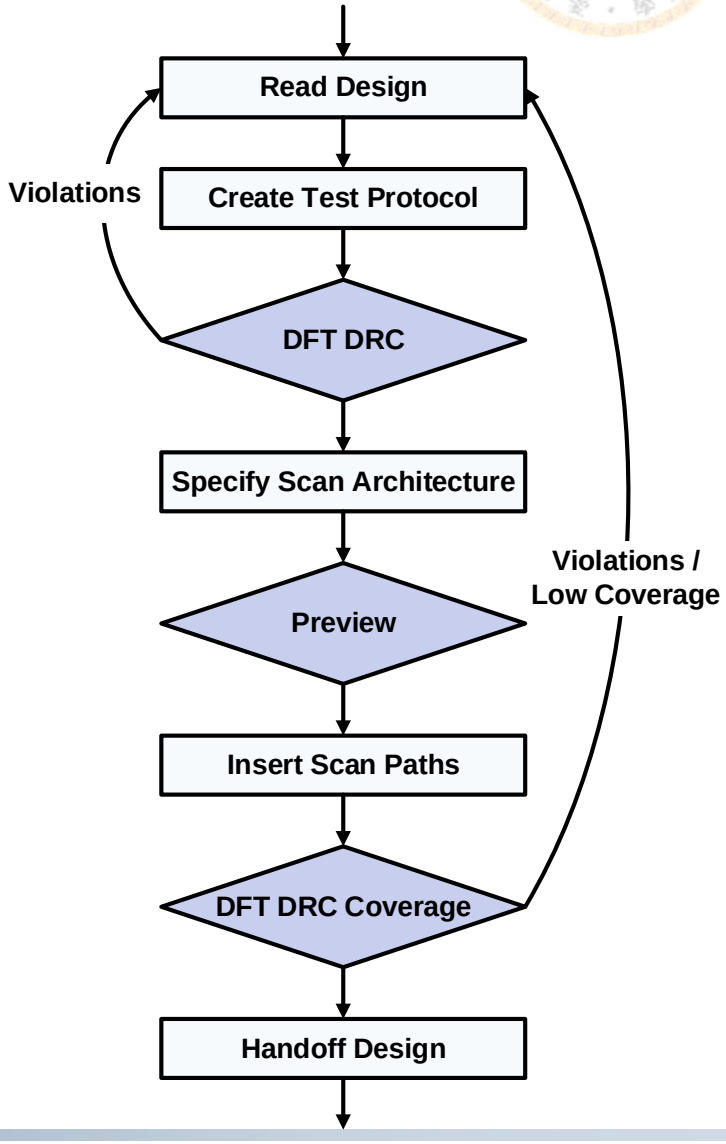
Scan Insertion Flow

- Pre-DFT DRC
  - Perform DRC checks prior to scan insertion
    - dft_drc
- Specify Scan Architecture
  - Number of scan chains, how clock domain are handled…
    - set_scan_configuration, set_scan_path



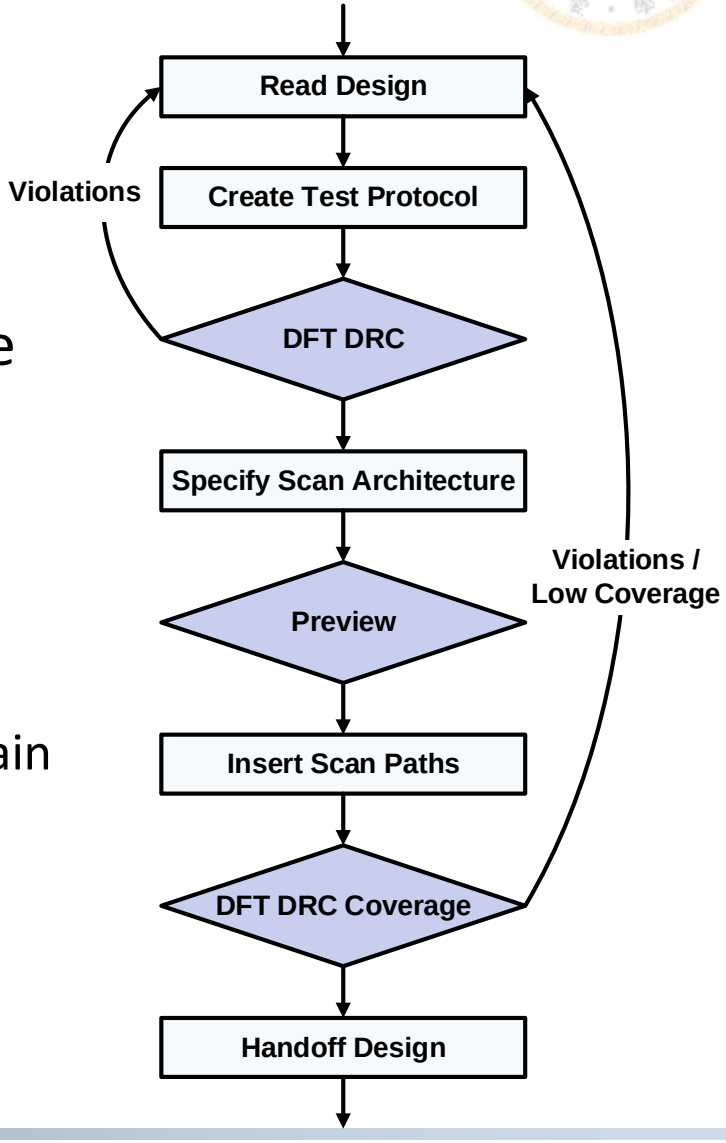
Scan Insertion Flow

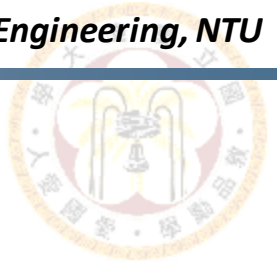
- Preview
  - Get a preview of the scan architecture before it is actually implemented in the design
    - preview_dft
- Insert Scan Path
  - Add scan circuit to the design
    - insert_dft



Scan Insertion Flow

- Post-DFT DRC
  - Validate the scan chains trace properly
  - Get ATPG coverage estimate for the scan inserted design
- Handoff Design
  - Write out files
    - DDC, verilog netlist, protocol file (for ATPG), SCANDEF (for backend scan chain reordering)…

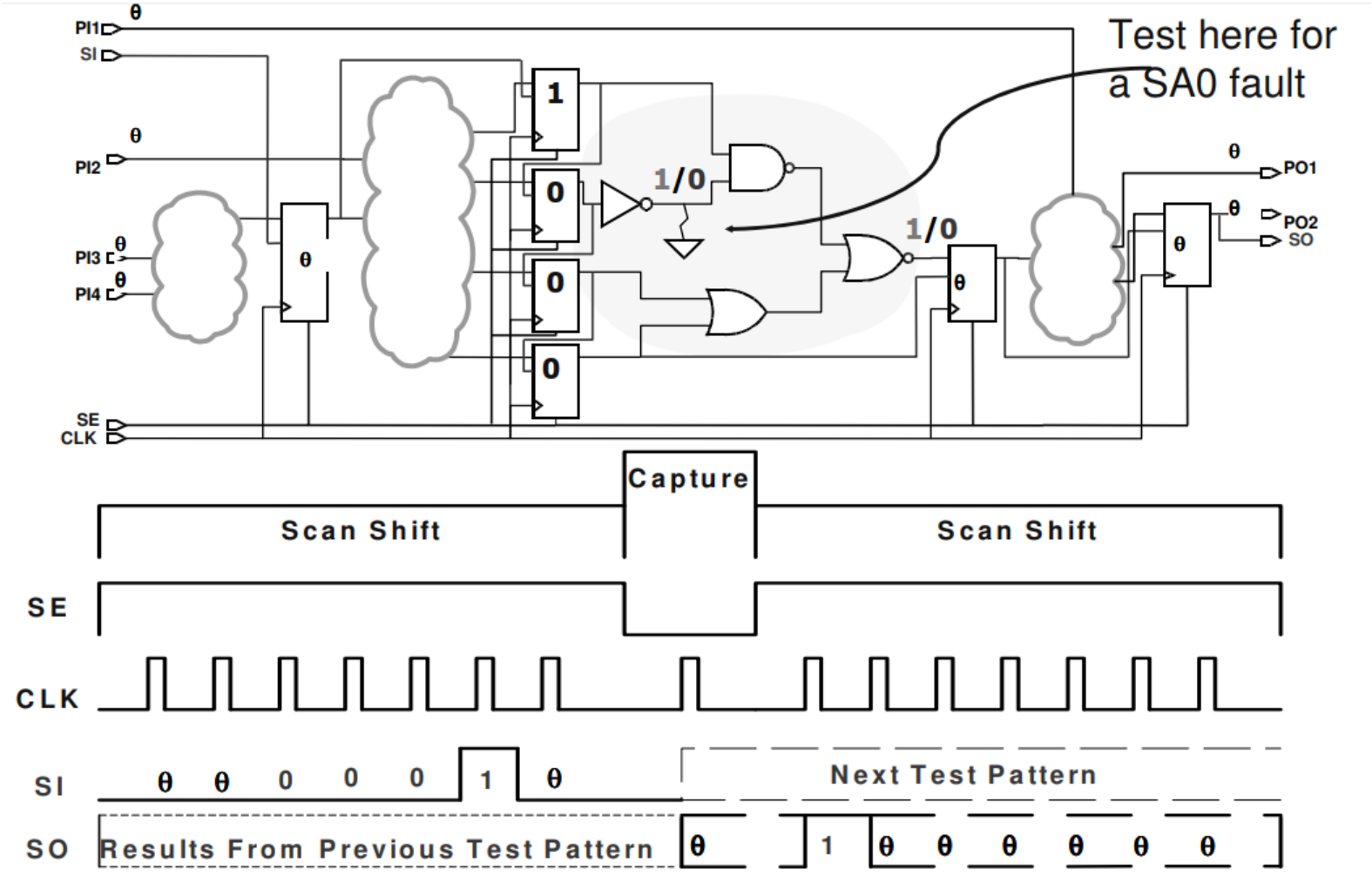


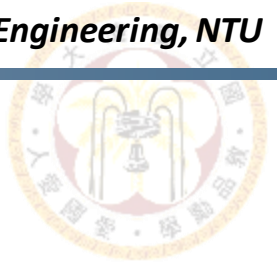


Scan Chain Testing Procedure

1. Set test mode signal, flip-flops accept data from input scan-in
2. Verify the scan path by shifting in and out test data
3. Set the shift register to an initial state
4. Apply a test pattern to the primary inputs of the circuit
5. Set normal mode, the circuit settles and can monitor the primary outputs of the circuit
6. Activate the circuit clock for one cycle
7. Return to test mode
8. Scan out the contents of the registers, simultaneously scan in the next pattern

Scan Chain Testing Procedure



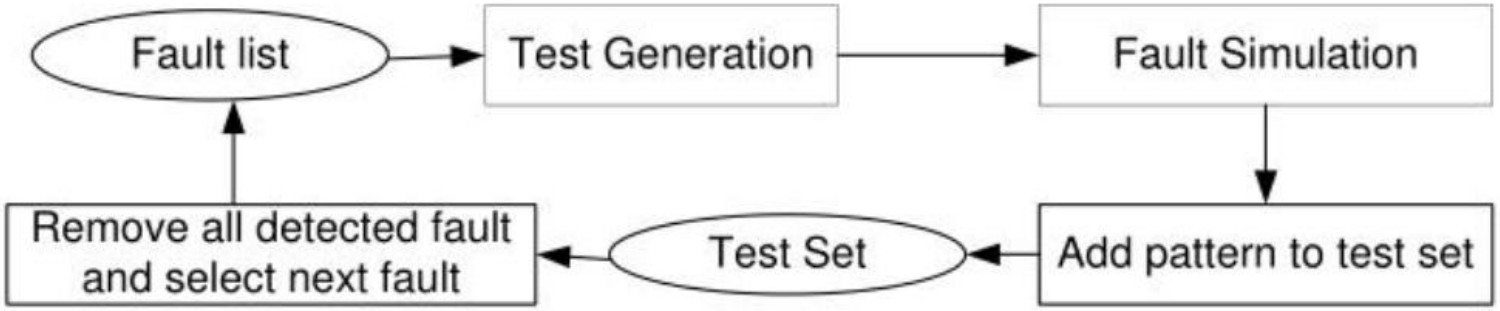


Outline

- Introduction
- Fault Modeling
- Design for Testability
- **Test Generation**
- Built-In Self-Test

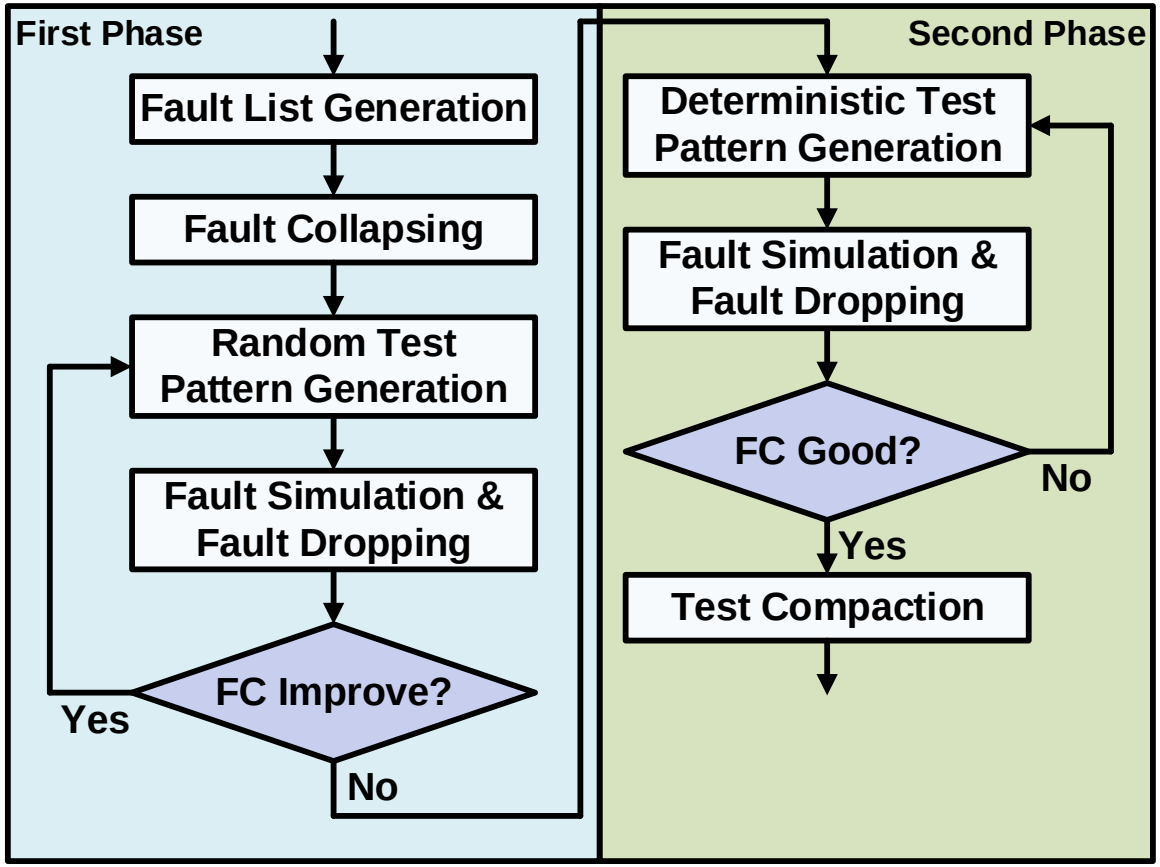
Automatic Test Pattern Generation (ATPG)

- Why ATPG
  - Generate high quality test patterns
  - Reduce human efforts
- Generate the test patterns for target fault model and keep the number of test patterns as small as possible



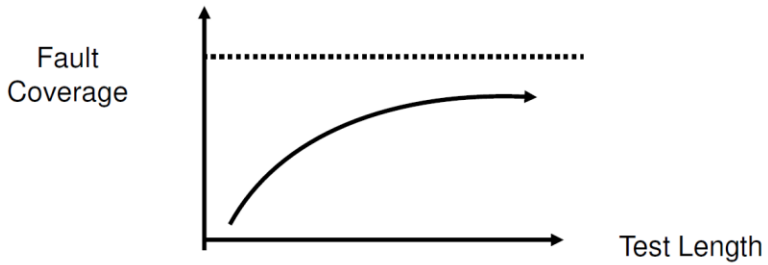
Typical ATPG Flow

- First phase: random test pattern generation
- Second phase: deterministic test pattern generation



Test Pattern Generation

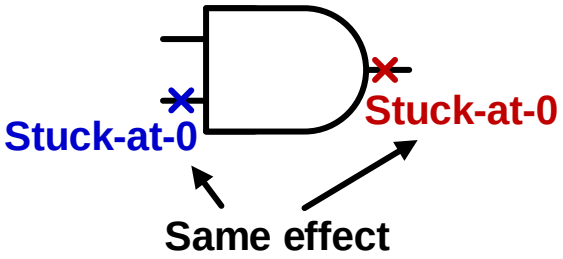
- Random test pattern generation
  - Idea: there are many easy-to-detect faults
    - Generating random patterns to achieve good initial coverage
  - Disadvantage: fault coverage often saturates after easy faults are detected



- Deterministic test pattern generation
  - Idea: used for faults that are missed by random test pattern generation
    - Choose a specific target fault and use algorithms such as D, PODEM, or FAN
  - Disadvantage: time consuming

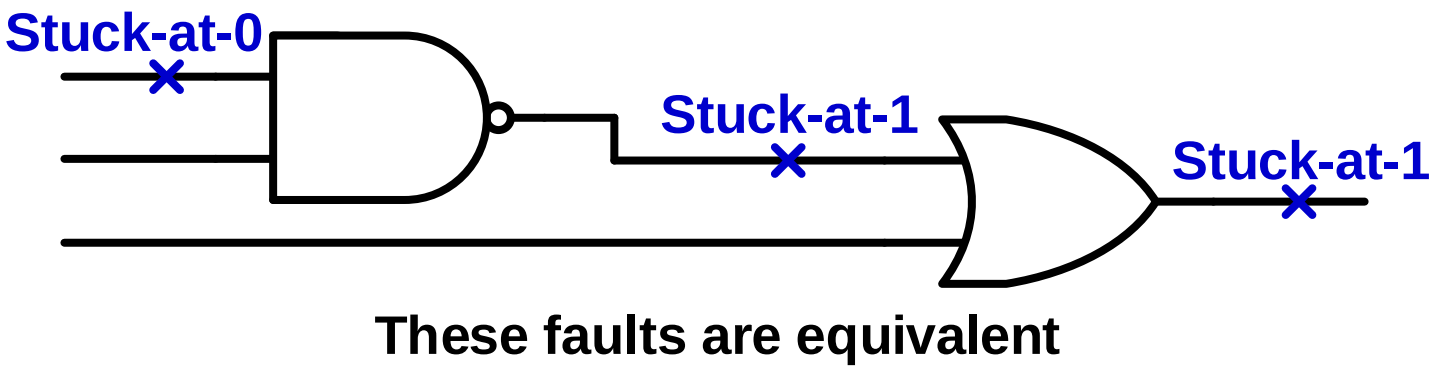
Fault Equivalence

- Two faults f1 and f2 are equivalent if all tests that detect f1 also detect f2
- AND gate: all stuck-at-0 faults are equivalent
- OR gate: all stuck-at-1 faults are equivalent
- NAND gate: all the input stuck-at-0 faults and the output stuck-at-1 faults are equivalent
- NOR gate: all the input stuck-at-1 faults and the output stuck-at-0 faults are equivalent
- Inverter
  - Input stuck-at-1 and output stuck-at-0 are equivalent
  - Input stuck-at-0 and output stuck-at-1 are equivalent



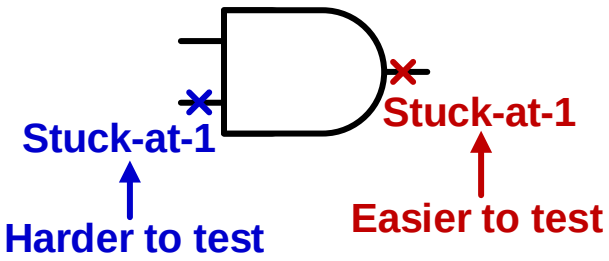
Equivalent Fault Group

- In a combinational circuit, many faults may form an equivalence group
- These equivalent faults can be found in a reversed topological order from POs to PIs



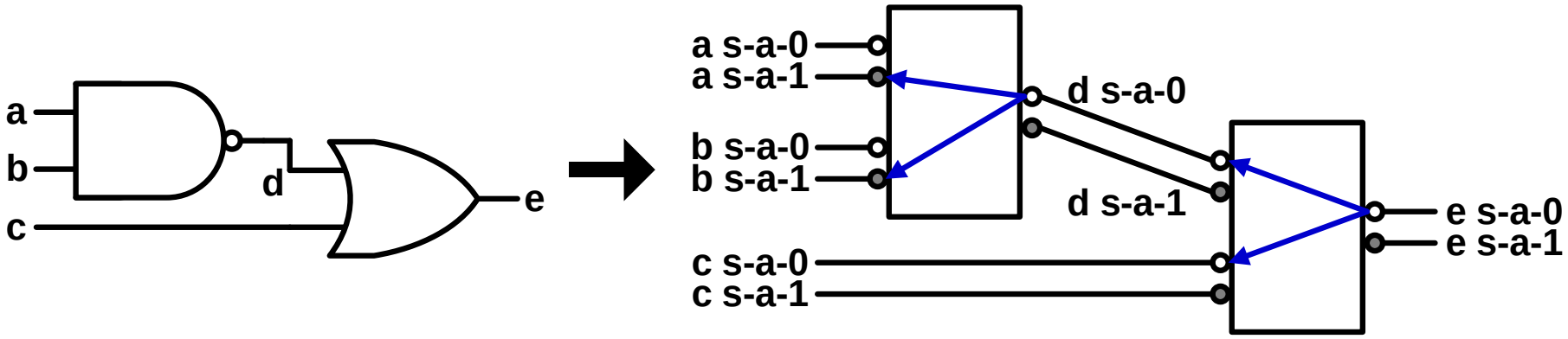
Fault Dominance

- All tests for fault f1 detect fault f2, meaning that f2 dominates f1
- If two faults dominate each other, they are considered equivalent
- AND gate: output stuck-at-1 dominates any input stuck-at-1
- NAND gate: output stuck-at-0 dominates any input stuck-at-1
- OR gate: output stuck-at-0 dominates any input stuck-at-0
- NOR gate: output stuck-at-1 dominates any input stuck-at-0



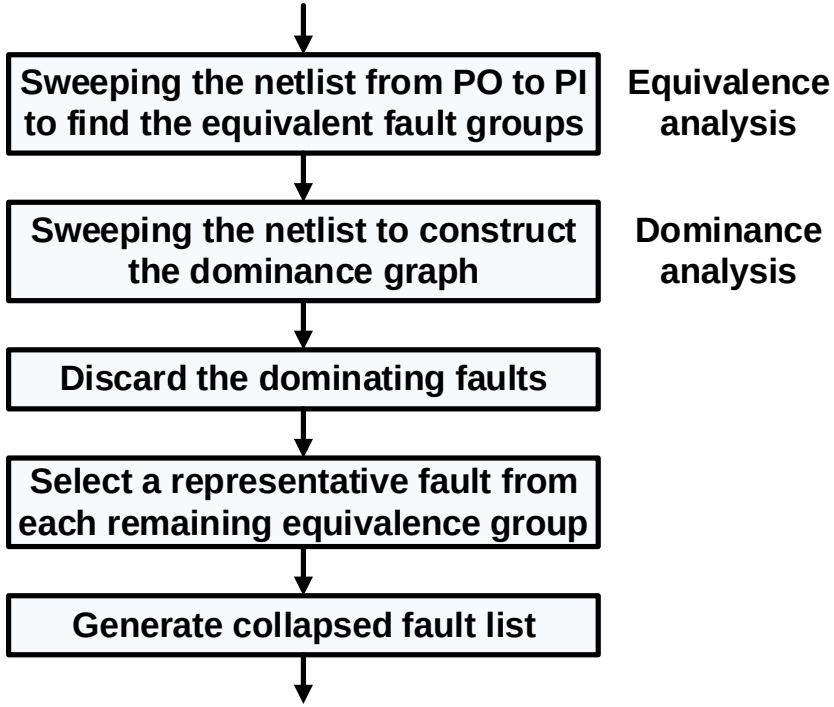
Dominance Graph

- Find out the transitive dominance relations among faults
- When fault f2 dominates fault f1, then an arrow is pointing from f2 to f1

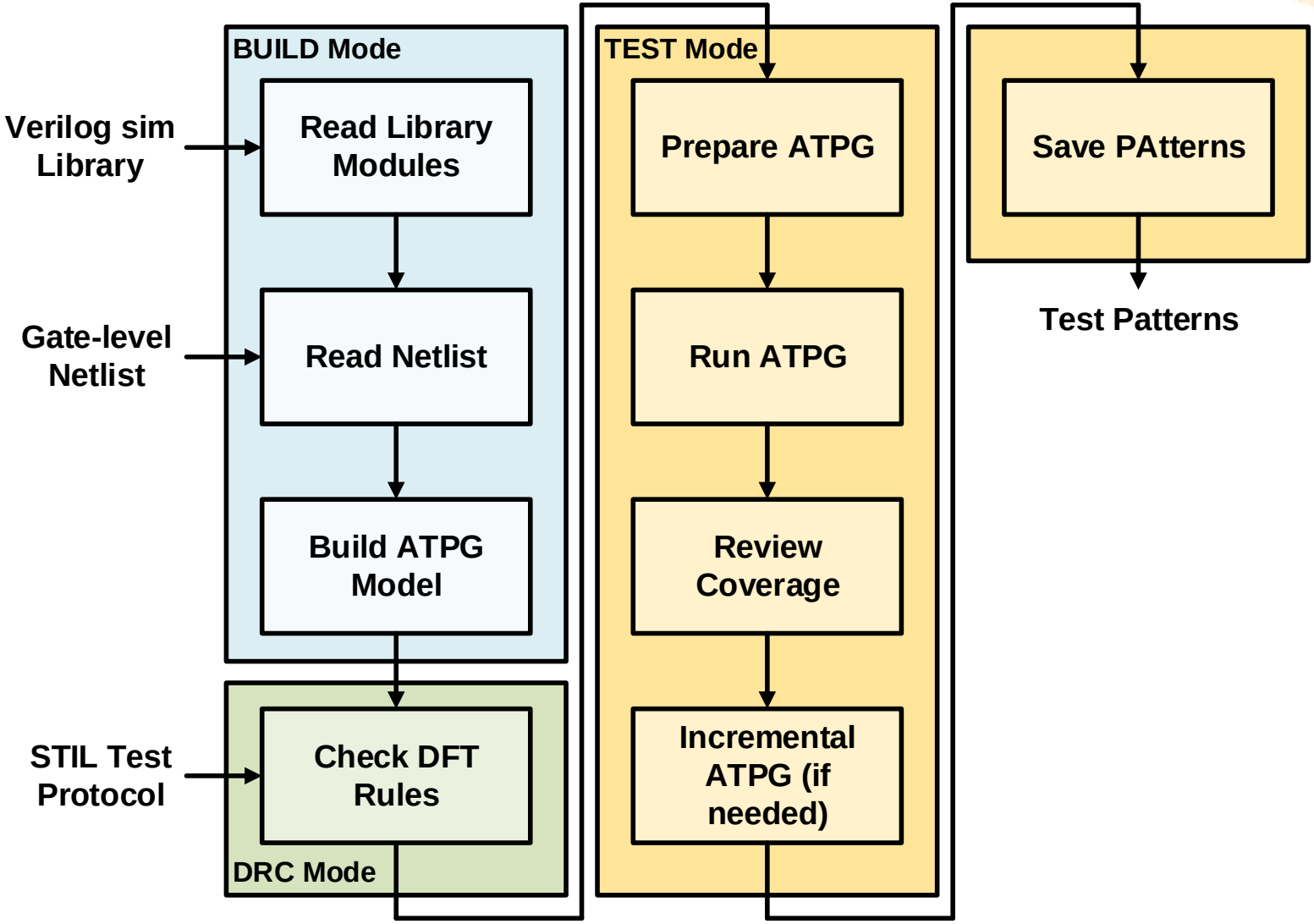


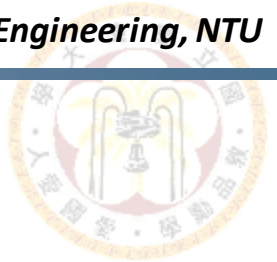
Fault Collapsing

- A technique for reducing the number of faults in circuits
  - Save memory and CPU time
  - Ease testing generation and fault simulation
- Collapse faults by fault equivalence and dominance



ATPG Flow on TestMAX ATPG





Launching TestMAX ATPG

- Launching TestMAX ATPG

```
tmax -nogui
```

```
tmax -shell
```

- Launching graphical user interface

```
tmax &
```

- Running script

```
tmax -shell script.tcl
```

```
tmax -nogui
tmax> source script.tcl
```

Example TestMAX ATPG Script

```
# Save transcript to log file
set_message –log lab.log –replace –level expert

# Reading libraries and design
read_netlist ../libs/cell.v
read_netlist ../design_data/design.v

# Run build
run_build_model

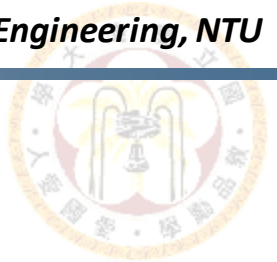
# Perform scan design rules check
run_drc ../design_data/design.spf

# Setup and run ATPG
add_faults –all

# Run ATPG engines
run_atpg –auto

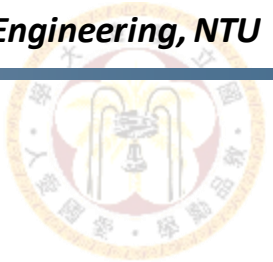
# Review test coverage
report_summaries
analyze_faults –class au
report_patterns –summary

# Save test patterns
write_patterns all_pats.stil –format stil -replace
```



Outline

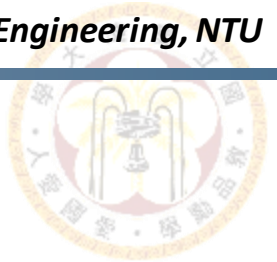
- Introduction
- Fault Modeling
- Design for Testability
- Test Generation
- **Built-In Self-Test**



BIST

- Embedding test circuitry to autonomously generate and apply test patterns, execute tests, and analyze results, all without the need for external test equipment

- Advantage
  - Reduce testing costs: minimize the need for expensive automated test equipment (ATE)
  - Automation
- Disadvantage
  - Area overhead
  - Performance degradation

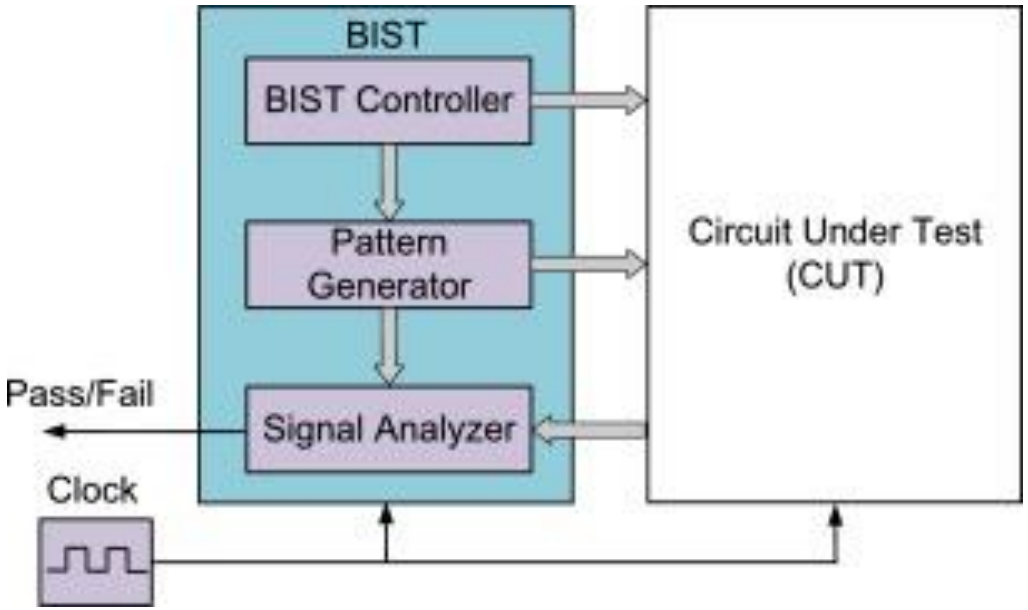


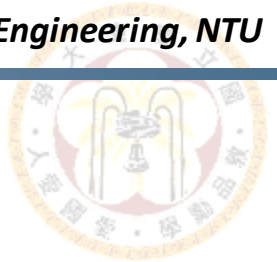
Types of BIST

- Logic BIST (LBIST)
  - Test the digital logic components
- Memory BIST (MBIST)
  - Test memory components like SRAM, DRAM, and Flash
- Analog BIST (ABIST)
  - Test analog components such as ADCs, DACs, and operational amplifiers

Components of BIST

- Hardware added to circuit under test (CUT)
  - Controller
  - (Test) Pattern generator
  - Signal analyzer





BIST Controller

- Coordinate the operations of different blocks of the BIST
- Receive a test mode input, which determines whether the system operates in normal mode or test mode

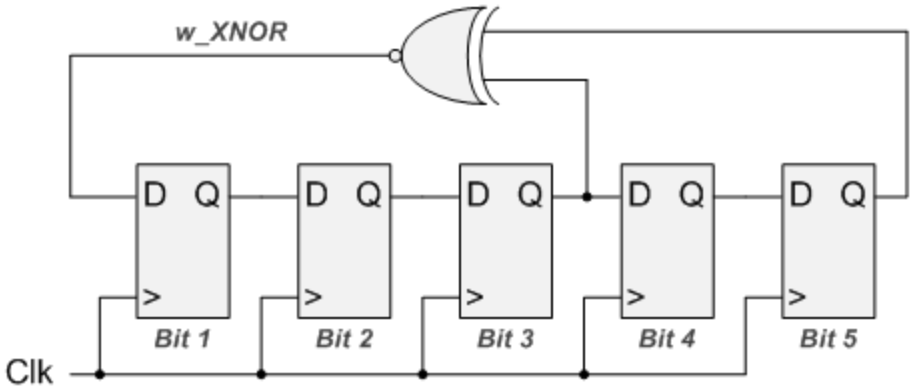
Pattern Generator

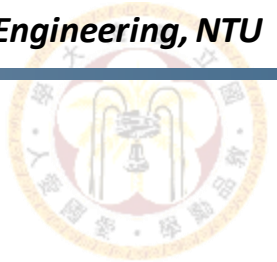
- Goal: Generate patterns on chip using minimum hardware, while achieving high fault coverage
- Common types
  - Linear feedback shift register (LFSR)
  - ROM-based pattern generator

Feature	LFSR	ROM-Based
Pattern Generation	Pseudorandom	Predifined
Flexibility	Depend on feedback taps	Can be customized
Test Coverage	Wide range of faults	Optimized for specific faults

LFSR

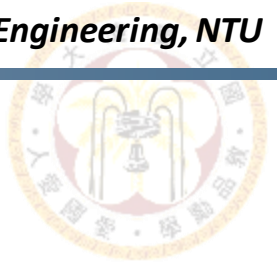
- Advantage
  - Simple and regular structure (only DFF and XNOR)
  - Compatible with scan DFT design
- If the LFSR stage is n, the maximum number of LFSR states is 2^n-1





Signal Analyzer

- Analyze the output responses from the CUT to determine if they match the expected results
- Goal: Compress the output response without losing too much accuracy
- Common types
  - Ones-counting compression
  - Transition-counting compression
  - Single input shift register (SISR)
  - Multiple input shift register (MISR)



Why Compression?

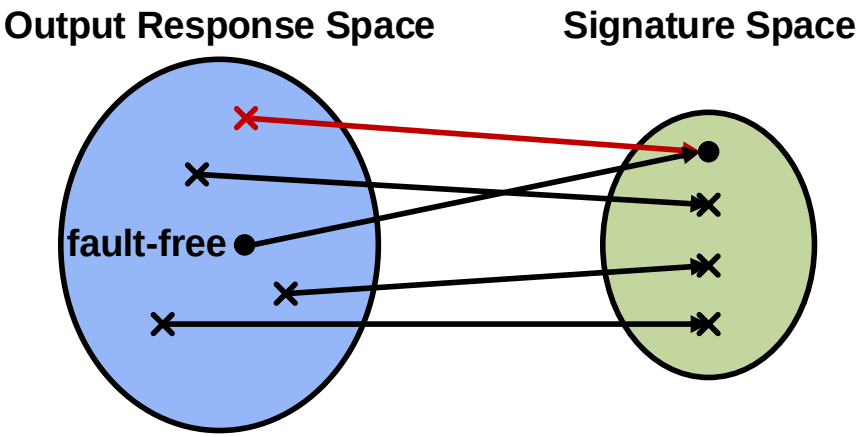
- Motivation: Bit-to-bit comparison is infeasible for BIST
- Signature analysis
  - Compress a very long output sequence into a single signature
  - Compare the compressed word with the pre-stored golden signature to determine the correctness of the circuit
- Problems
  - Many output sequences may have the same signature after the compression leading to the aliasing problem
  - Poor diagnosis resolution after compression

Aliasing Effect in Response Compression

- Aliasing: a faulty response is mapped to the same signature as the fault-free circuit
- Response compression is a mapping from the output response space to the signature space
- Probability of aliasing (PAL)

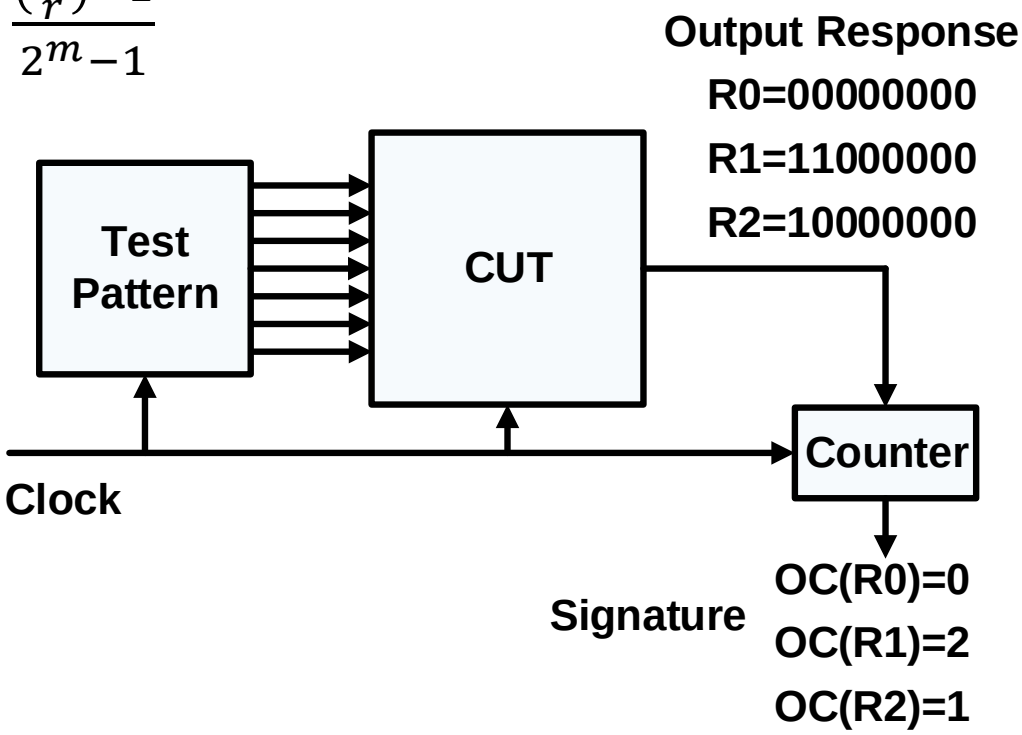
$$\text{PAL} = \frac{\text{Number of faulty outputs that generated gold signature}}{\text{Total number of faulty outputs}}$$

- In this example, PAL = 1/4 = 25%



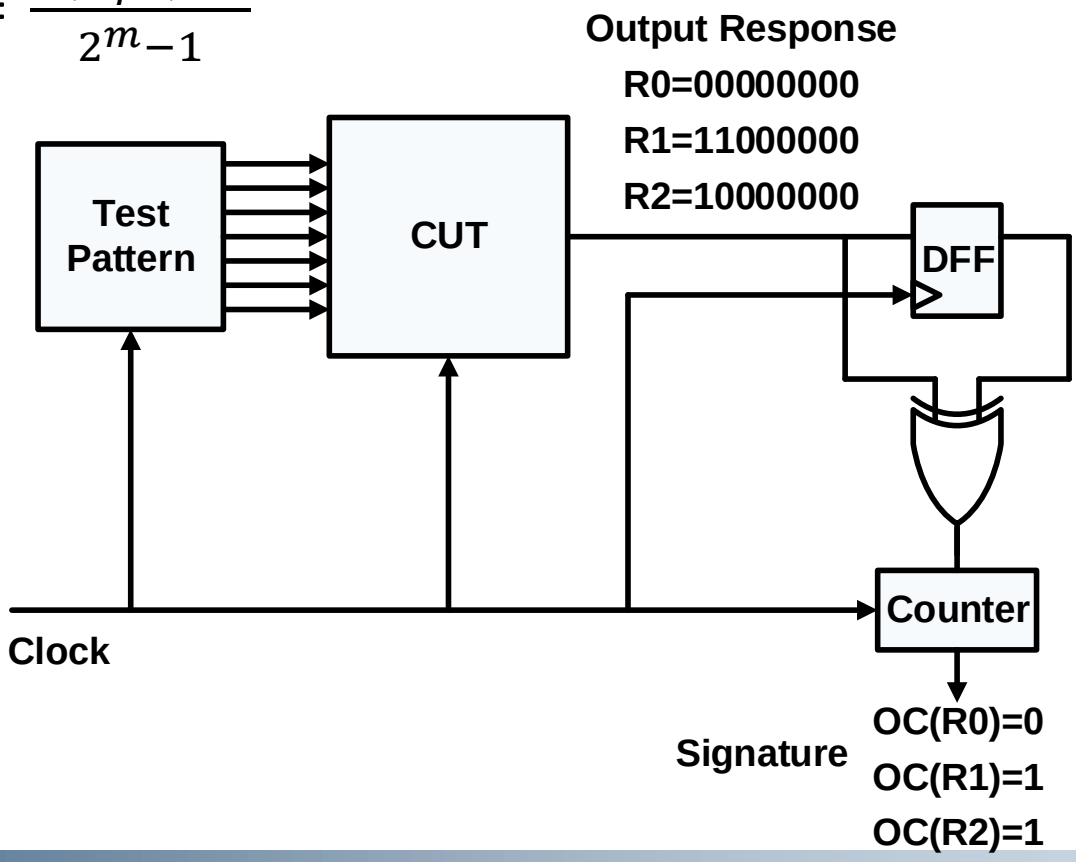
Ones-Counting Compression

- Count the number of ones in the output sequence
- Aliasing when fault-free and faulty signatures are the same
- If sequence length of output response is *m* and signature is *r*, then PAL = $\frac{\binom{m}{r}-1}{2^m-1}$



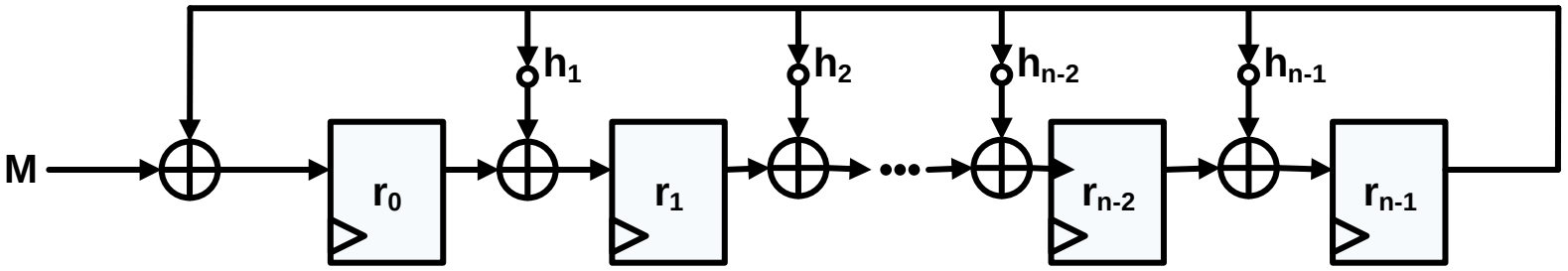
Transition-Counting Compression

- Count the number of 0->1 and 1->0 transitions
- If sequence length of output response is *m* and signature is *r*, then PAL = $\frac{2\binom{m-1}{r}-1}{2^m-1}$

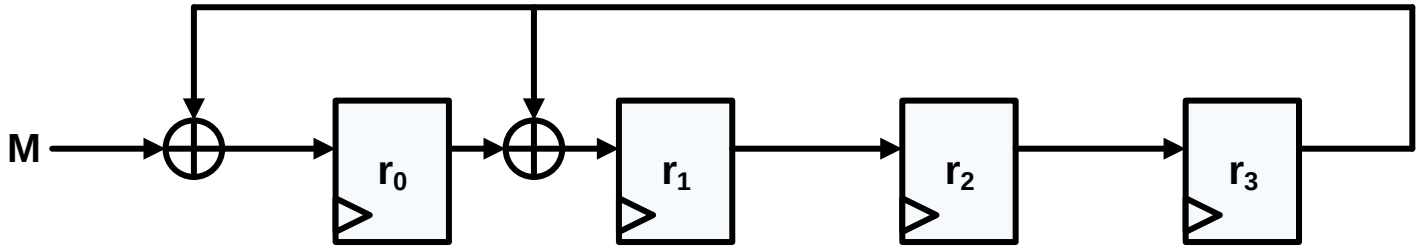


Single Input Shift Register (SISR)

- The signature is the contents of shift registers after the last input bit has been sampled
- If sequence length of output response is *m* and LFSR has *n* stages
  - Number of bit streams with same signature = $\frac{2^m}{2^n} = 2^{m-n}$
  - PAL = $\frac{2^{m-n}-1}{2^m-1} \approx 2^{-n}$
  - If all error streams are equally likely, PAL depends only on the number of LFSR stages



Example: 4-Stage SISR



M	r_0	r_1	r_2	r_3
1	**0**	**0**	**0**	**0**
1	1	0	0	0
0	1	1	0	0
1	0	1	1	0
1	1	0	1	1
0	0	0	0	1
0	1	1	0	0
1	0	1	1	0
S	**1**	**0**	**1**	**1**

Fault-Free Signature

M'	r_0	r_1	r_2	r_3
1	**0**	**0**	**0**	**0**
1	1	0	0	0
0	1	1	0	0
1	0	1	1	0
0	1	0	1	1
0	1	0	0	1
1	1	0	0	0
1	1	1	0	0
S'	**1**	**1**	**1**	**0**

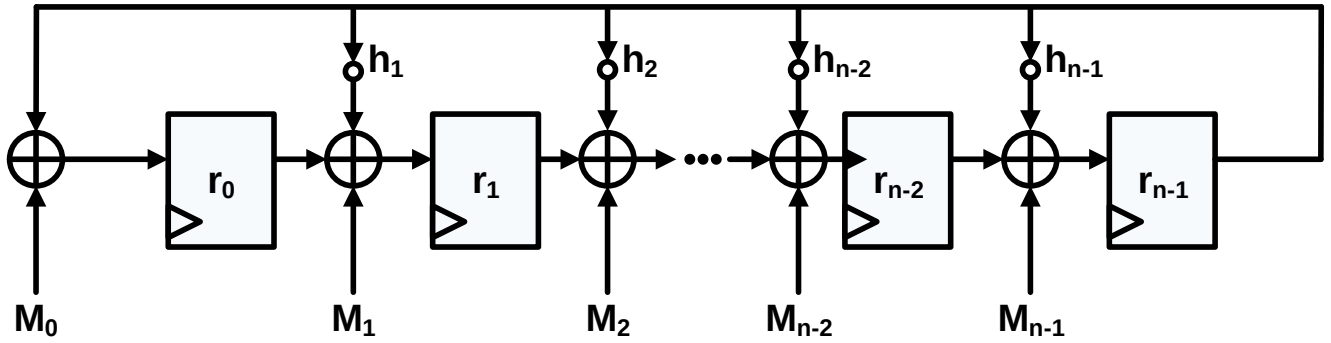
Signature for Fault f1

M''	r_0	r_1	r_2	r_3
1	**0**	**0**	**0**	**0**
0	1	0	0	0
1	0	1	0	0
1	1	0	1	0
0	1	1	0	1
0	1	0	1	0
1	0	1	0	1
1	0	1	1	0
S''	**1**	**0**	**1**	**1**

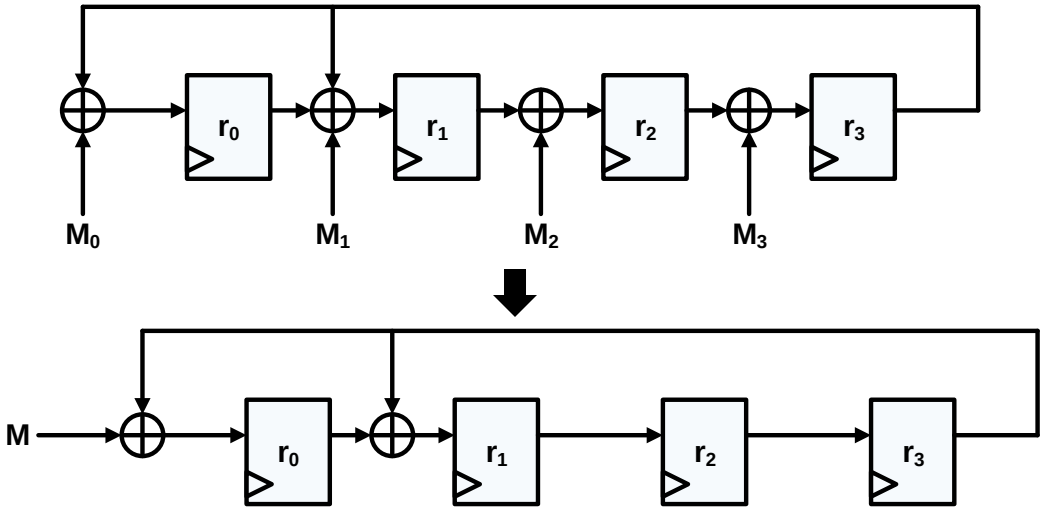
Signature for Fault f2

Multiple Input Shift Register (MISR)

- An n-input MISR can be remodeled as a single-input SISR



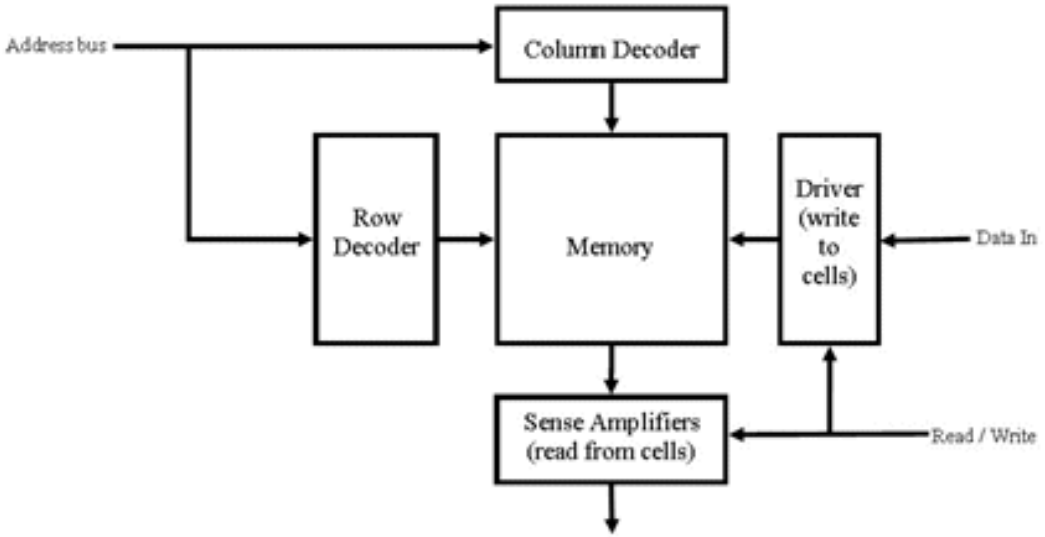
- Example: 4-stage MISR



M_0	1	0	0	1	0			
M_1		0	1	0	1	0		
M_2			1	1	0	0	0	
M_3				1	0	0	1	0
M	1	0	0	1	1	0	1	1

Memory Testing

- Memory Specific Challenges
  - High density of cells
  - More sensitive to influences of neighbors, noises
- Memory Fault Models
  - Stuck-at Faults (SAF)
  - Stuck-Open Faults (SAF)
  - Transition Faults (TF)
  - Address Faults (AF)
  - Coupling Faults (CF)
  - Neighborhood Pattern Sensitive Faults (NPSF)

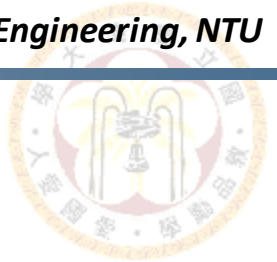


Source: https://www.einfochips.com

Basic Algorithms: MSCAN & Checkerboard

- MSCAN / zero-one algorithm
  - 1. Write 0 to every cell -> Read 0 from every cell
  - 2. Write 1 to every cell -> Read 1 from every cell
  - Target: Basic Stuck-at Faults and Address Decoder Faults
- Checkerboard algorithm
  - 1. Write checkerboard pattern to all cells (e.g., 010101...)
  - 2. Read checkerboard pattern from all cells
  - 3. Write inverse checkerboard pattern to all cells
  - 4. Read inverse checkerboard pattern from all cells
  - Target: Bridging faults between adjacent cells



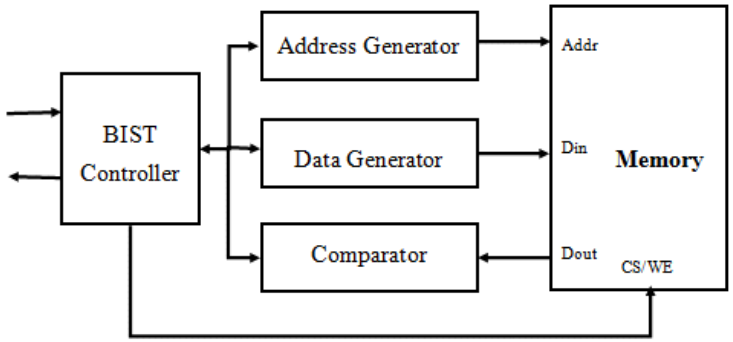


March Test Algorithms

- Concept: "Marching" through memory addresses systematically to detect complex faults.
- Modified Algorithmic Test Sequence (MATS)
  - 1. Write 0s with to all cells (to initialize)
  - 2. Read 0s, then write 1s with up addressing order
  - 3. Read 1s, with down addressing order (Also write 0s in MATS+)
- Other march test algorithms
  - MATS++, March X, March C, SMarchCKBD
- Target: Stuck-At, Transition, Address faults, Coupling faults.

MBIST Hardware Architecture

- BIST Controller
  - A FSM that determines current state (e.g., Read/Write)
- Address Generator
  - Up/Down Counter driven by the Controller
- Data Generator
  - Generates expected data (0, 1, or checkerboard patterns)
- Comparator
  - Compare Dout from RAM with expected value



Source: https://www.einfochips.com